

Module - Plateformes de Développement

Spring

Support de cours

Par :

Dr. Ing. Faouzi JAIDI

faouzi.jaidi@gmail.com

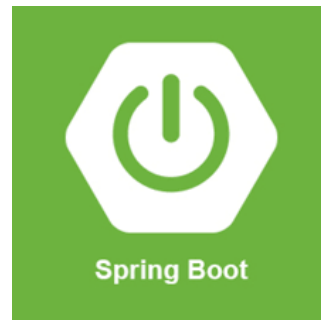
2021-2022

Plan

- 1. Introduction
- 2. Maven
- 3. Logs : Log4J
- 4. Tests: JUnit
- 5. Tests & Qualité : Sonar
- 6. IOC & ID
- 7. Spring Boot
- 8. Spring Data JPA
- 9. Spring MVC REST
- 10. JSF
- 11. AOP
- 12. Projet
- 13. Spring Security
- 14. Extensions

Spring

Spring Data JPA



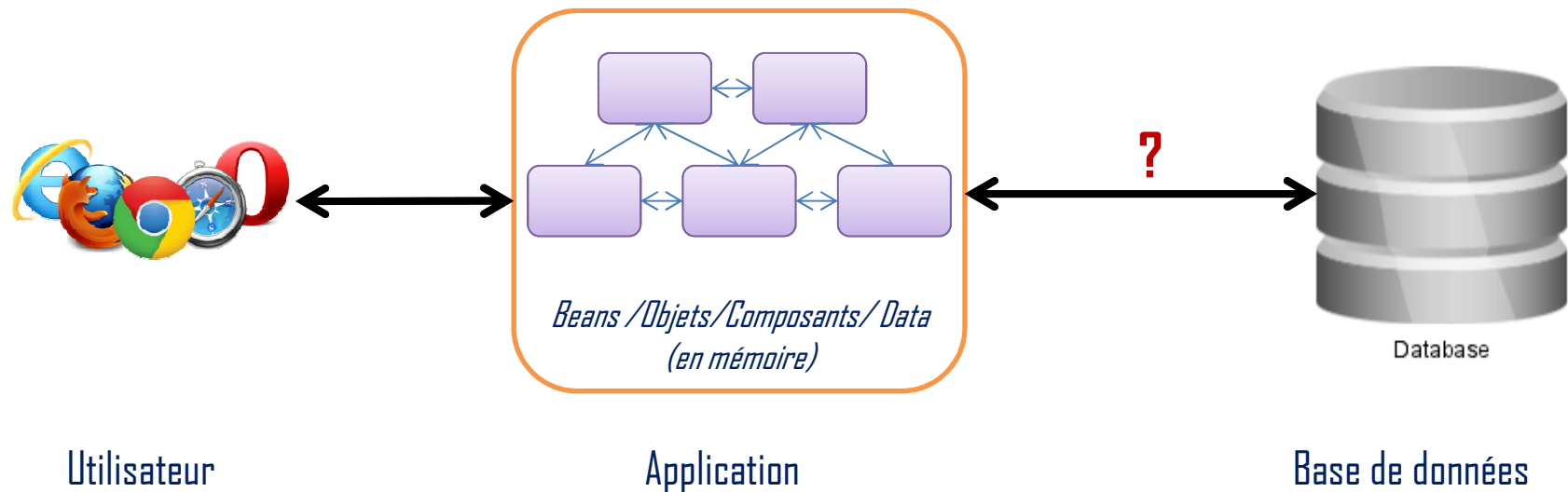


Partie 1 :

Concepts et Architecture

Persistence et Mapping Objet Relationnel

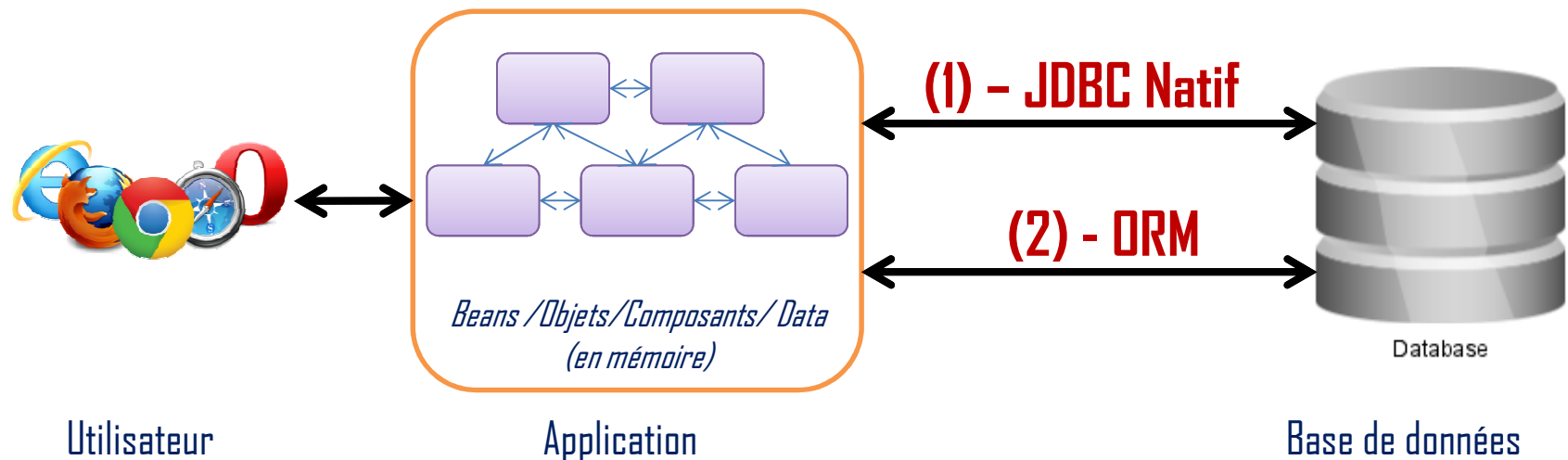
□ Persistence [*Persistence*]



Comment stocker les objets modélisés en mémoire dans la base de données? Et comment les récupérer?

Persistence et Mapping Objet Relationnel

- La gestion de la **persistence** des données/d'objets métiers: un mécanisme responsable de la sauvegarde et de la restauration des données/des objets.



1. Utilisation du JDBC natif : développer manuellement le code de projection des objets sur le support relationnel.
2. Utilisation d'un ORM (Framework de projection objet-relationnel)

Persistence et Mapping Objet Relationnel

□ Utilisation du JDBC Natif

○ *JDBC*

- *JDBC : Java DataBase Connectivity*
- Un framework JAVA (ensemble de classes et d'interfaces),
- Une interface de programmation créée par Sun Microsystems,
- Une API (Application Programming Interface) pour les programmes utilisant la plateforme Java,
- Elle permet aux applications Java d'accéder à et d'interagir avec des bases de données, en utilisant des pilotes (Drivers) JDBC
- Les fournisseurs des SGBD fournissent des drivers JDBC.

Persistence et Mapping Objet Relationnel

○ Inconvénients

- Problème de lisibilité:
 - Pas de séparation entre le code technique et le code métier.
 - Problème de maintenance:
 - Utilisation du langage SQL rend la couche d'accès aux données difficilement maintenable.
- *Le concept d'ORM a vu le jour pour éviter les inconvénients liés à l'utilisation de l'API JDBC nativement.*

Persistence et Mapping Objet Relationnel

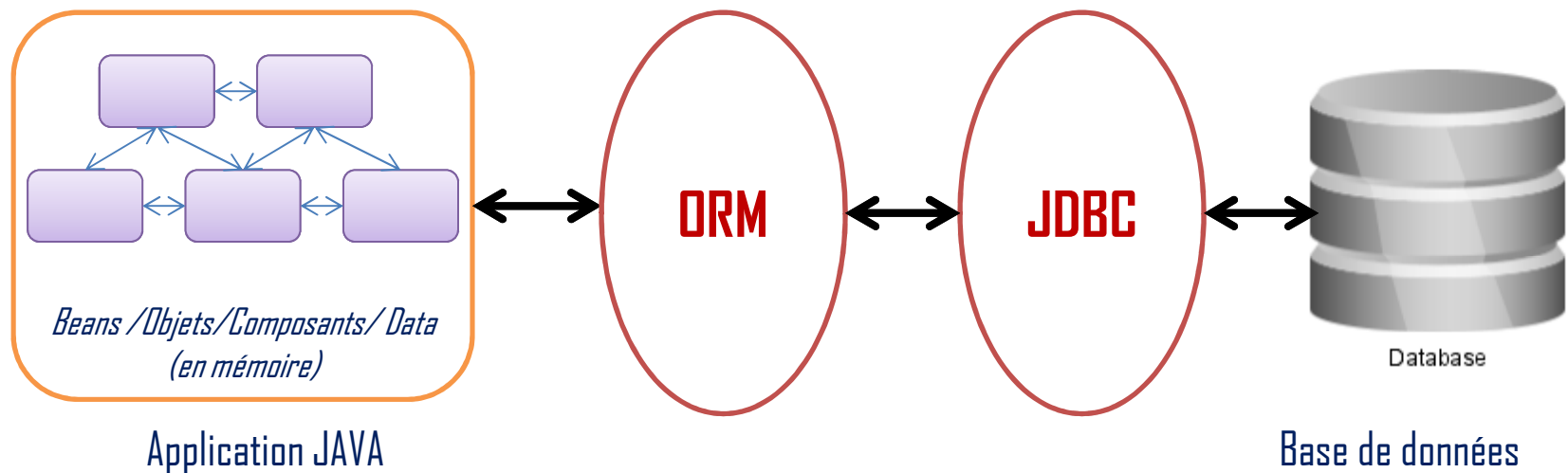
□ Utilisation d'un ORM

○ *ORM*

- *ORM: Object-Relational Mapping*
- Une technique basée sur la création d'une couche d'abstraction entre le monde objet et monde relationnel,
- Définit des correspondances entre les schémas de la base de données et les classes (objets) du programme applicatif,
- Une technique de programmation qui permet d'associer une ou plusieurs classes avec une table, et chaque attribut de la classe avec un champ de la table.

Persistence et Mapping Objet Relationnel

o Principe



- Exemples d'ORM: Hibernate, Toplink, EclipseLink, Spring Data JPA, ...
- Ces frameworks implémentent la spécification **JavaEE JPA**.

Persistence et Mapping Objet Relationnel

□ JPA

- *JPA: Java Persistence API*
- Un ensemble de spécifications pour la normalisation et la standardisation de la communication avec la base de données,
- Une interface de programmation Java permettant de normaliser l'utilisation et la communication avec la couche de données d'une application Java,
- L'utilisation de JPA nécessite un fournisseur de persistance (ORM) implémente les spécifications JPA.
- Nous allons utiliser **Spring Data JPA** qui se base sur **Hibernate** comme implémentation de **JPA**.

Persistence et Mapping Objet Relationnel

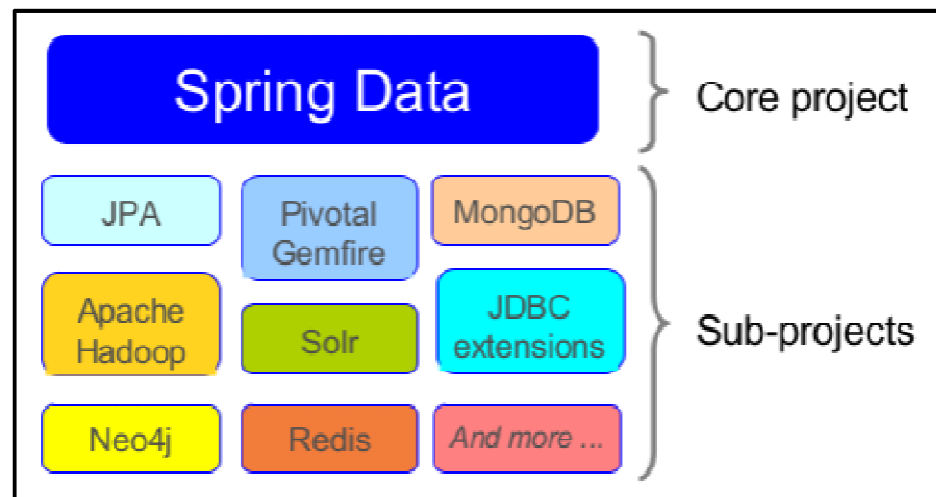
□ Hibernate

- Un framework open source / un projet maintenu par l'entreprise JBoss appartenant à RedHat,
- Un fournisseur de persistance (ORM),
- Hibernate gère le mapping entre les objets de l'application et la base de données. Ce mapping permet d'assurer la transformation d'objets vers la base de données et vice versa.
- L'utilisation d'hibernate permet d'optimiser le temps de développement:
 - Génération automatique des tables à partir des entités Java.
 - Génération automatique du code SQL (CRUD).

Persistence et Mapping Objet Relationnel

□ Spring Data JPA

○ *Spring DATA*

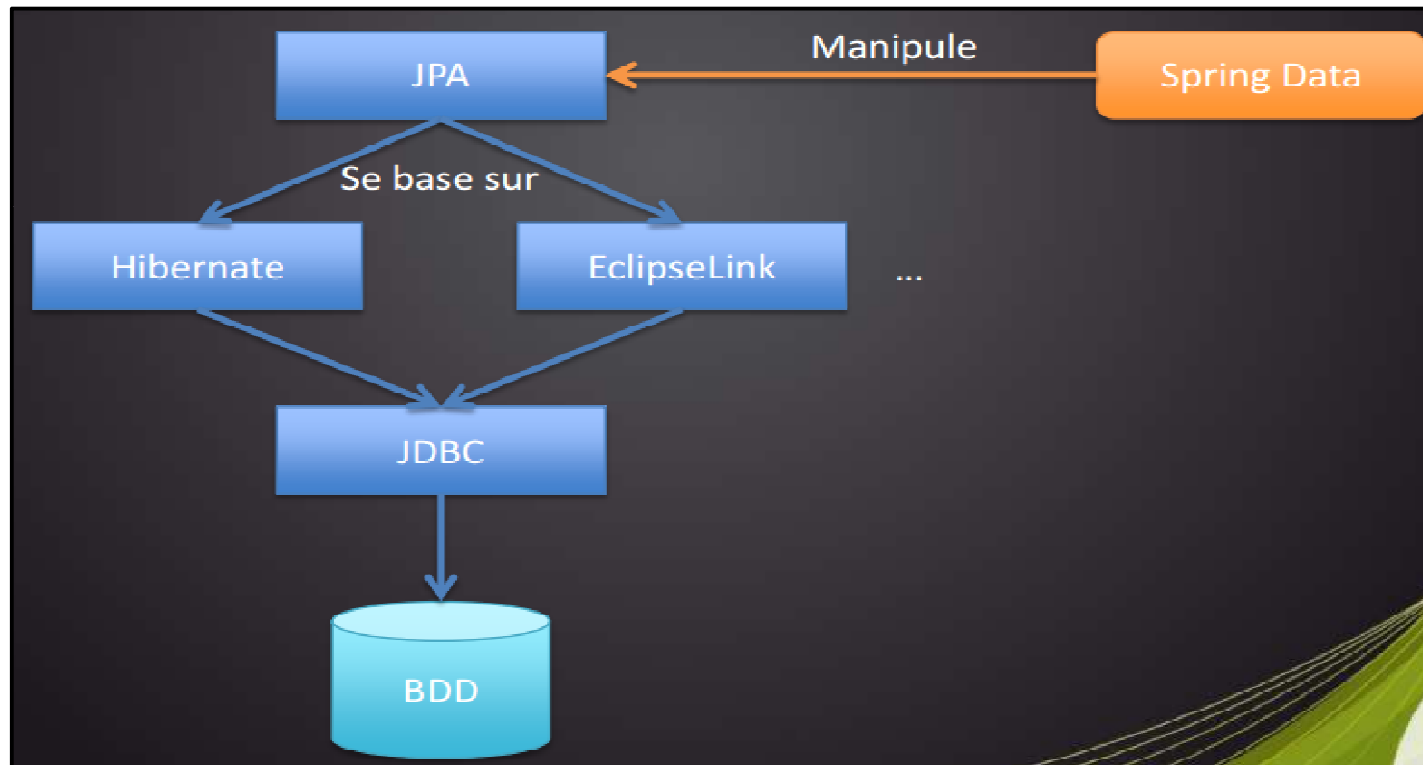


- Spring Data est un projet Spring qui a pour objectif de simplifier l'interaction avec différents systèmes de stockage de données :
 - Facilite l'écriture des couches d'accès aux données.
 - Offre une abstraction commune pour l'accès aux données quelle que soit la source de données (SQL ou NoSQL).

Persistence et Mapping Objet Relationnel

o *Spring DATA JPA*

- Un sous projet du projet Spring Data.

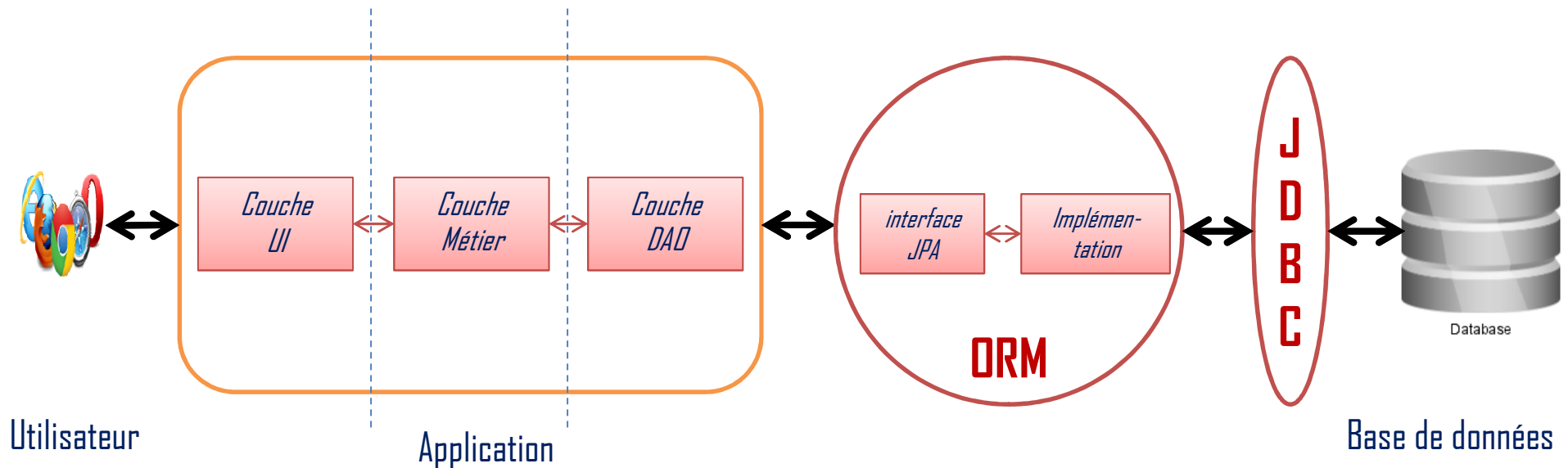


Persistence et Mapping Objet Relationnel

- Spring Data JPA:
 - Un module qui permet d'interagir avec une base de données relationnelle en représentant les objets du domaine métier sous la forme **d'entités JPA**.
 - Fournit l'interface *JpaRepository<T, ID>* qui hérite de *CrudRepository<T, ID>* et qui fournit un ensemble de méthodes plus spécifiquement adaptées pour interagir avec une base de données relationnelle.
- L'interface *CrudRepository<T, ID>* hérite de *Repository<T, ID>* et fournit un ensemble d'opérations élémentaires pour la manipulation des objets.

Persistence et Mapping Objet Relationnel

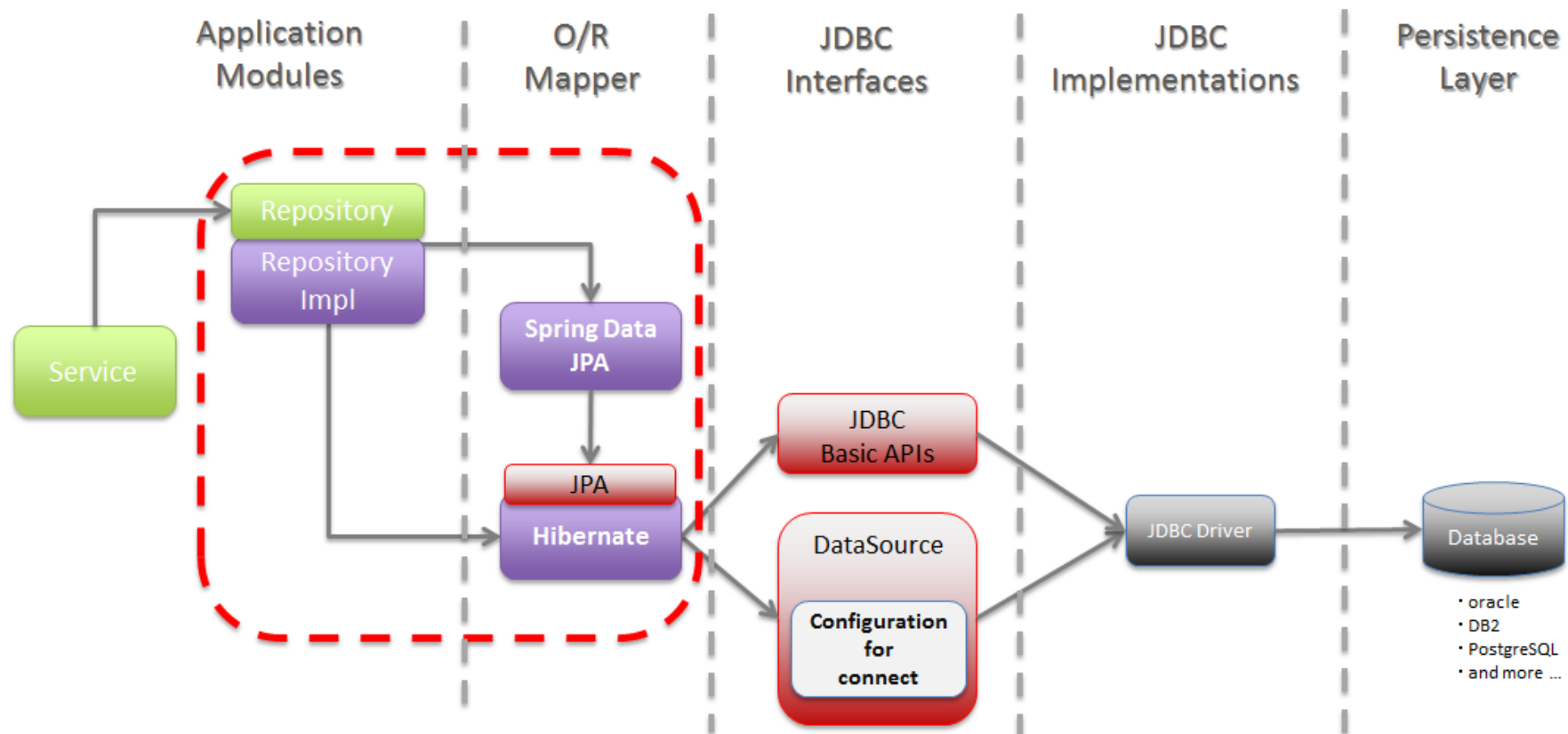
□ Architecture



- Offrir une abstraction commune pour l'accès aux données quelle que soit la source de données (SQL ou NoSQL).

Persistence et Mapping Objet Relationnel

□ Architecture





Partie 2 :

Entité JPA

Entité JPA

□ Entité

○ Description

- Une entité est :
 - une classe dont les instances peuvent être persistantes,
 - un simple POJO (*Plain Old Java Object*),
 - un JavaBean (attributs, constructeurs, accesseurs, mutateurs),
 - un POJO mappé vers une table de la base de données grâce à des méta donnée via l'API *Java Persistence* est nommé **bean entité** (*Entity bean*).
- Les entités dans les spécifications de l'API *Java Persistence* permettent d'encapsuler les données d'une occurrence d'une ou plusieurs tables.

Entité JPA

o Caractéristiques

- Une entité JPA doit respecter les règles suivantes:
 - Être déclaré avec l'annotation **@Entity**.
 - Possède au moins une propriété déclarée comme identité de l'entité avec l'annotation **@Id**.
 - Implémente l'interface **java.io.Serializable**.
- JPA propose **plusieurs annotations**, pour supporter un mapping O/R (entre un bean entité et une table) assez complet, à utiliser:
 - sur la classe pour la correspondance avec la table associée.
 - sur les attributs pour la correspondance avec les colonnes de la table.

Entité JPA

□ Annotations de Base

○ *@Entity* [Obligatoire]

- Permet de définir une classe Java comme étant une entité persistante et donc associée à une table dans la base de données.

○ *@Table* [Facultatif]

- Permet de préciser le nom de la table concernée par le mapping.
- Par défaut, l'entité est liée à la table de la base de données correspondant au nom de la classe de l'entité.

<i>Attributs</i>	<i>Rôle</i>
<i>name</i>	<i>Nom de la table</i>
<i>catalog</i>	<i>Catalogue de la table</i>
<i>schema</i>	<i>Schéma de la table</i>
<i>uniqueConstraints</i>	<i>Contraintes d'unicité sur une ou plusieurs colonnes</i>

Entité JPA

- *@Id* [Obligatoire]

- Permet de déclarer une des propriétés de la classe comme étant la clé primaire de la table.

- *@GeneratedValue* [Facultatif]

- Permet de générer automatiquement la clé primaire.
- Cette annotation possède plusieurs attributs :

<i>Attributs</i>	<i>Rôle</i>
<i>strategy</i>	<i>Précise le type de générateur à utiliser : TABLE, SEQUENCE, IDENTITY ou AUTO. La valeur par défaut est AUTO.</i>
<i>generator</i>	<i>Nom du générateur à utiliser</i>

Entité JPA

- L'attribut "strategy" obligatoire:
- **strategy = GenerationType.AUTO** : le fournisseur de persistance (hibernate) est chargé de la génération de la clé primaire. Une séquence unique de clé primaire pour **tout le schéma de base de données** défini dans une table *hibernate_sequence*.
- **strategy = GenerationType.TABLE** : le fournisseur de persistance (hibernate) est chargé de la génération de la clé primaire. Une séquence de clés primaires **par table** définie dans une table *hibernate_sequences* qui contient deux colonnes : une pour le nom de la séquence et l'autre pour la prochaine valeur.
- **strategy = GenerationType.IDENTITY** : le mécanisme propre au SGBD est chargé de la génération de la clé primaire. Par exemple: MySQL utilise l'option *AUTO-INCREMENT*, Postgres ou Oracle utilisent une séquence.
- **strategy = GenerationType.SEQUENCE** : équivalente à GenerationType.AUTO, mais c'est le développeur qui définit les détails de la séquence.

Entité JPA

- `@Column` [Facultatif]

- Permet d'indiquer le nom de la colonne dans la table quand elle est différente du nom de la propriété en java.
- Cette annotation possède plusieurs attributs :

Attributs	Rôle
name	Nom de la colonne
table	Nom de la table dans le cas d'un mapping multi-table
unique	Indique si la colonne est unique
nullable	Indique si la colonne est nullable
insertable	Indique si la colonne doit être prise en compte dans les requêtes de type insert
updatable	Indique si la colonne doit être prise en compte dans les requêtes de type update
columnDefinition	Précise le DDL de définition de la colonne
length	Indique la taille d'une colonne de type chaîne de caractères
precision	Indique la taille d'une colonne de type numérique
scale	Indique la précision d'une colonne de type numérique

Entité JPA

- *@Transient* [Facultatif]

- Permet de demander de ne pas tenir compte du champ lors du mapping.
- Indique que l'attribut ne sera pas mappé (et donc non persisté) dans la table.

- *@Temporal*

- Permet de fournir des informations complémentaires sur la façon dont les propriétés encapsulant **des données temporelles** (*Date et Calendar*) sont associées aux colonnes dans la table (*date, time ou timestamp*).
- La valeur par défaut est *timestamp*.
 - TIME : 30-09-19 10:50:56.780000000 AM
 - DATE : 30-09-19
 - TIMESTAMP : 1569840656 (nombre de secondes entre 01/01/1970 et la date voulue)

Entité JPA

o *@Enumerated*

- Permet d'associer une propriété de type **énumération** à une colonne de la table sous la forme d'un numérique ou d'une chaîne de caractères.
- Le paramètre **EnumType** peut avoir comme valeur **EnumType.ORDINAL** (valeur par défaut) ou **EnumType.STRING**.

```
@Enumerated(EnumType.STRING)  
private Role role;
```

```
public enum Role {  
    Etudiant, Employé, Enseignant  
}
```

Entité JPA

□ Objets Embarqués

- JPA permet d'utiliser dans les entités des objets Java qui ne sont pas des entités mais qui sont agrégés dans l'entité et dont les propriétés seront mappées sur les colonnes correspondantes dans la table.

○ *@Embeddable*

- La classe embarquée est un simple POJO qui doit être marquée avec l'annotation ***@javax.persistence.Embeddable***.

○ *@Embedded*

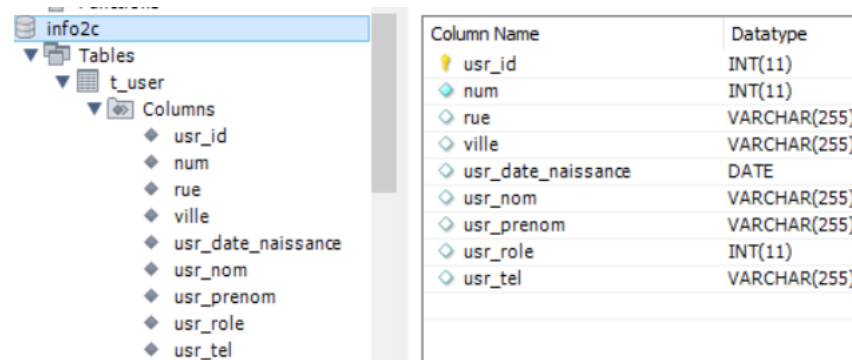
- Dans l'entité, il faut utiliser l'annotation ***@javax.persistence.Embedded*** sur la propriété du type de la classe embarquée.

Entité JPA

0 Exemple

```
@Embeddable
public class Adresse {
    private int num;
    private String rue;
    private String ville;
    public int getNum() {
        return num;
    }
    //...
}
```

```
@Entity
@Table(name = "T_USER")
public class User implements Serializable{
    //...
    private Adresse adresse;
    @Embedded
    public Adresse getAdresse() {
        return adresse;
    }
    //...
}
```



Column Name	Datatype
usr_id	INT(11)
num	INT(11)
rue	VARCHAR(255)
ville	VARCHAR(255)
usr_date_naissance	DATE
usr_nom	VARCHAR(255)
usr_prenom	VARCHAR(255)
usr_role	INT(11)
usr_tel	VARCHAR(255)

Entité JPA

□ Clé Composite

- Un identifiant [eq. une clé primaire] peut être composée de plusieurs attributs [eq. colonnes].
- JPA propose deux façons de gérer ce cas de figure avec les annotations :
 - *@javax.persistence.IdClass*
 - *@javax.persistence.EmbeddedId* et *javax.persistence.Embeddable;*
- La clé primaire doit être représentée par une classe Java dont les attributs correspondent aux composants de la clé primaire. Cette classe doit être sérialisable, posséder un constructeur sans paramètre, et fournir une implémentation des méthodes equals() et hashCode().

Entité JPA

○ 1^{er} cas :

- Utilisation de `@IdClass`:
 - La classe entité est annotée par `@IdClass` qui prend en paramètre le nom de la classe clé primaire.
 - La classe clé primaire n'est pas annotée,
 - Les attributs de la classe clé primaire ont les mêmes noms et mêmes types que les attributs annotés par `@Id` dans la classe entité.

Entité JPA

```
@Entity
@Table(name = "T_USER_CASE1")
@IdClass(UserPK.class)
public class User_CompKey implements
Serializable{
    private static final long
serialVersionUID = 1L;

    @Id
    @Column(name = "USR_NUM")
    private int num;

    @Id
    @Column(name = "USR_NOM")
    private String nom;

    @Column(name = "USR_PRENOM")
    private String prenom;

    @Column(name = "USR_TEL")
    private String tel;

    @Embedded
    private Adresse adresse;

    @Temporal(TemporalType.DATE)
    @Column(name = "USR_DATE_NAISSANCE")
    private Date date_naissance;

    @Transient
    private float age;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

```
public class UserPK implements Serializable {
    private static final long serialVersionUID = 1L;

    private int num;
    private String nom;

    // Constructeur, getters, setters, equals et
    hashCode

}
```



Column Name	Datatype	PK	NN	UQ
usr_nom	VARCHAR(255)	☒	☒	☐
usr_num	INT(11)	☒	☒	☐
num	INT(11)	☐	☒	☐
rue	VARCHAR(255)	☐	☐	☐
ville	VARCHAR(255)	☐	☐	☐
usr_date_naissance	DATE	☐	☐	☐
usr_prenom	VARCHAR(255)	☐	☐	☐
role	VARCHAR(255)	☐	☐	☐
usr_tel	VARCHAR(255)	☐	☐	☐

Entité JPA

0 2^{er} cas :

- Utilisation de *@Embeddable* et *@EmbeddedId*:
 - La classe clé primaire est annotée par *@Embeddable*.
 - L'attribut identifiant de la classe entité est annoté avec l'annotation *@EmbeddedId*.

Entité JPA

```
@Entity
@Table(name = "T_USER_CASE2")
public class User2_CompKey implements
Serializable{
    private static final long
serialVersionUID = 1L;

    @EmbeddedId
    private UserPK_EMB id;

    @Column(name = "USR_PRENOM")
    private String prenom;

    @Column(name = "USR_TEL")
    private String tel;

    @Column(name = "USR_ADRESSE")
    @Embedded
    private Adresse adresse;

    @Temporal(TemporalType.DATE)
    @Column(name = "USR_DATE_NAISSANCE")
    private Date date_naissance;

    @Transient
    private float age;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

```
@Embeddable
public class UserPK_EMB implements Serializable{
    private static final long serialVersionUID = 1L;

    @Column(name = "USR_NUM")
    private int num;

    @Column(name = "USR_NOM")
    private String nom;
    // Constructeur, getters, setters, equals et hashCode
}
```



Column Name	Datatype	PK	NN	UQ
usr_nom	VARCHAR(255)	☒	☒	☐
usr_num	INT(11)	☒	☒	☐
num	INT(11)	☐	☒	☐
rue	VARCHAR(255)	☐	☐	☐
ville	VARCHAR(255)	☐	☐	☐
usr_date_naissance	DATE	☐	☐	☐
usr_prenom	VARCHAR(255)	☐	☐	☐
role	VARCHAR(255)	☐	☐	☐
usr_tel	VARCHAR(255)	☐	☐	☐

Entité JPA

□ Héritage

- JPA propose trois stratégies pour mapper une hiérarchie de classes:

○ *SINGLE_TABLE* [Stratégie par défaut]:

- Une seule table est utilisée pour toutes les classes de la hiérarchie,
- Les attributs de la hiérarchie des entités est mis à plat dans une seule table.
 - *@Inheritance*: sur la classe mère pour spécifier la stratégie.
 - *@DiscriminatorColumn*: sur la classe mère pour préciser la colonne qui servira de discriminant.
 - *@DiscriminatorValue*: sur chaque classe de la hiérarchie pour préciser la valeur dans la colonne discriminant.

Entité JPA

```
@Entity
@Table(name = "T_HERITAGE_SINGLE_TABLE")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "USR_TYPE",
discriminatorType = DiscriminatorType.STRING)
@DiscriminatorValue("Parent")
public class Heritage_Parent implements Serializable{
private static final long serialVersionUID = 1L;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "HP_ID")
private int id;

@Column(name = "HP_ATT_PARENT")
private String att_parent;
}
```

```
@Entity
@DiscriminatorValue("FILS")
public class Heritage_Fils
extends Heritage_Parent
implements Serializable{
private static final long
serialVersionUID = 1L;

@Column(name = "HF_ATT_FILS")
private String att_fils;
}
```



t_heritage_single_table		Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G
Columns	usr_type	usr_type	VARCHAR(31)	☐	☒	☐	☐	☐	☐	☐	☐
	hp_id	hp_id	INT(11)	☒	☒	☐	☐	☐	☐	☒	☐
	hp_att_parent	hp_att_parent	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐
	hf_att_fils	hf_att_fils	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐
				☐	☐	☐	☐	☐	☐	☐	☐

Entité JPA

o *TABLE_PER_CLASS* :

- Une table par classe concrète.
- Chaque entité concrète de la hiérarchie est associée à une table.
 - *@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)*: sur la classe mère pour spécifier la stratégie.
 - Cette stratégie de mapping limite le choix dans la stratégie de génération des identifiants à utiliser.
 - Il n'est donc pas possible d'utiliser les stratégies AUTO et IDENTITY.

Entité JPA

```
@Entity
@Table(name = "T_HERITAGE_PARENT_TABLE_PER_CLASS")
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Heritage2_Parent implements Serializable{
    private static final long serialVersionUID = 1L;

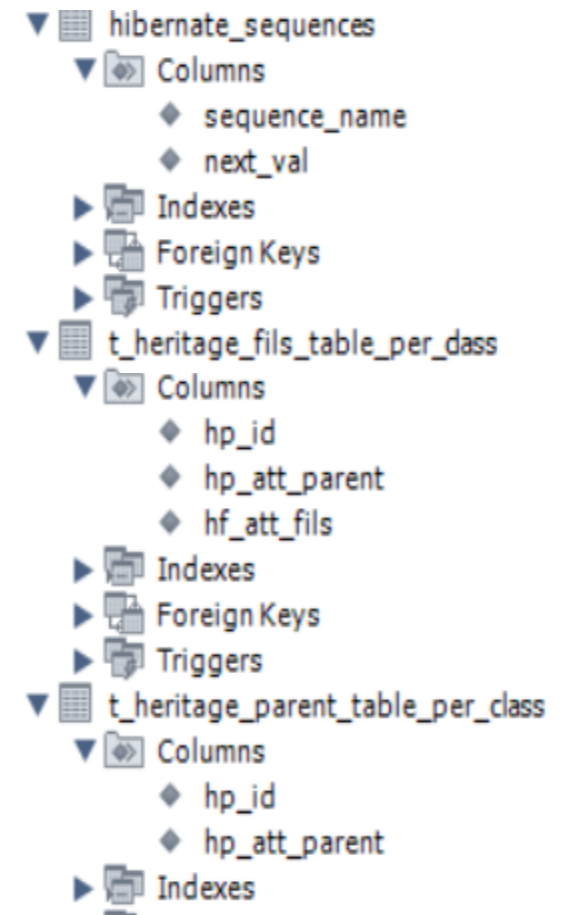
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    @Column(name = "HP_ID")
    private int id;

    @Column(name = "HP_ATT_PARENT")
    private String att_parent;
}
```

```
@Entity
@Table(name = "T_HERITAGE_FILS_TABLE_PER_CLASS")

public class Heritage2_Fils extends Heritage2_Parent
implements Serializable{
    private static final long serialVersionUID = 1L;

    @Column(name = "HF_ATT_FILS")
    private String att_fils;
}
```



Entité JPA

o *JOINED* :

- Une table est utilisée pour chaque classe de la hiérarchie.
- Chaque entité de la hiérarchie, concrète ou abstraite, est associée à sa propre table.
- Une table pour chaque classe fille, plus une table pour la classe parente. Une jonction est alors nécessaire pour instancier la classe fille.
- *@Inheritance(strategy = InheritanceType.JOINED)*: sur la classe mère pour spécifier la stratégie.
- *@PrimaryKeyJoinColumn*: sur chaque classe fille pour préciser la colonne qui est la clé primaire et qui servira comme clé étrangère lors de la jointure avec la table de la classe mère.

Entité JPA

```
@Entity
@Table(name = "T_HERITAGE_PARENT_JOINED")
@Inheritance(strategy = InheritanceType.JOINED)

public class Heritage3_Parent implements Serializable{
    private static final long serialVersionUID = 1L;

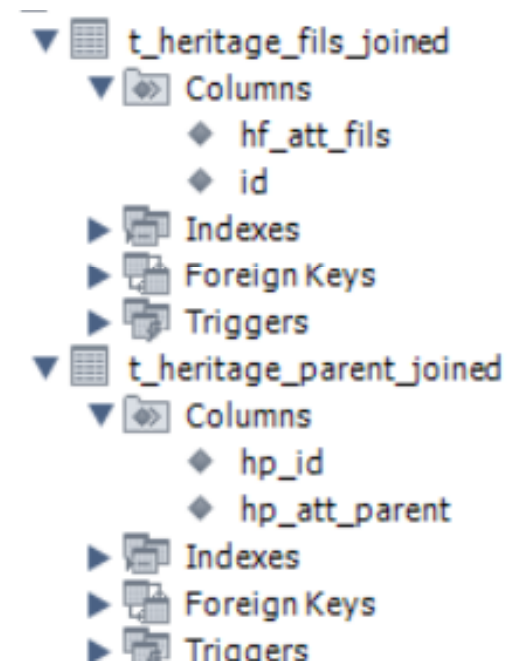
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    @Column(name = "HP_ID")
    private int id;

    @Column(name = "HP_ATT_PARENT")
    private String att_parent;
}
```

```
@Entity
@Table(name = "T_HERITAGE_FILS_JOINED")
@PrimaryKeyJoinColumn(name = "id")

public class Heritage3_Fils extends Heritage3_Parent
implements Serializable{
    private static final long serialVersionUID = 1L;

    @Column(name = "HF_ATT_FILS")
    private String att_fils;
}
```



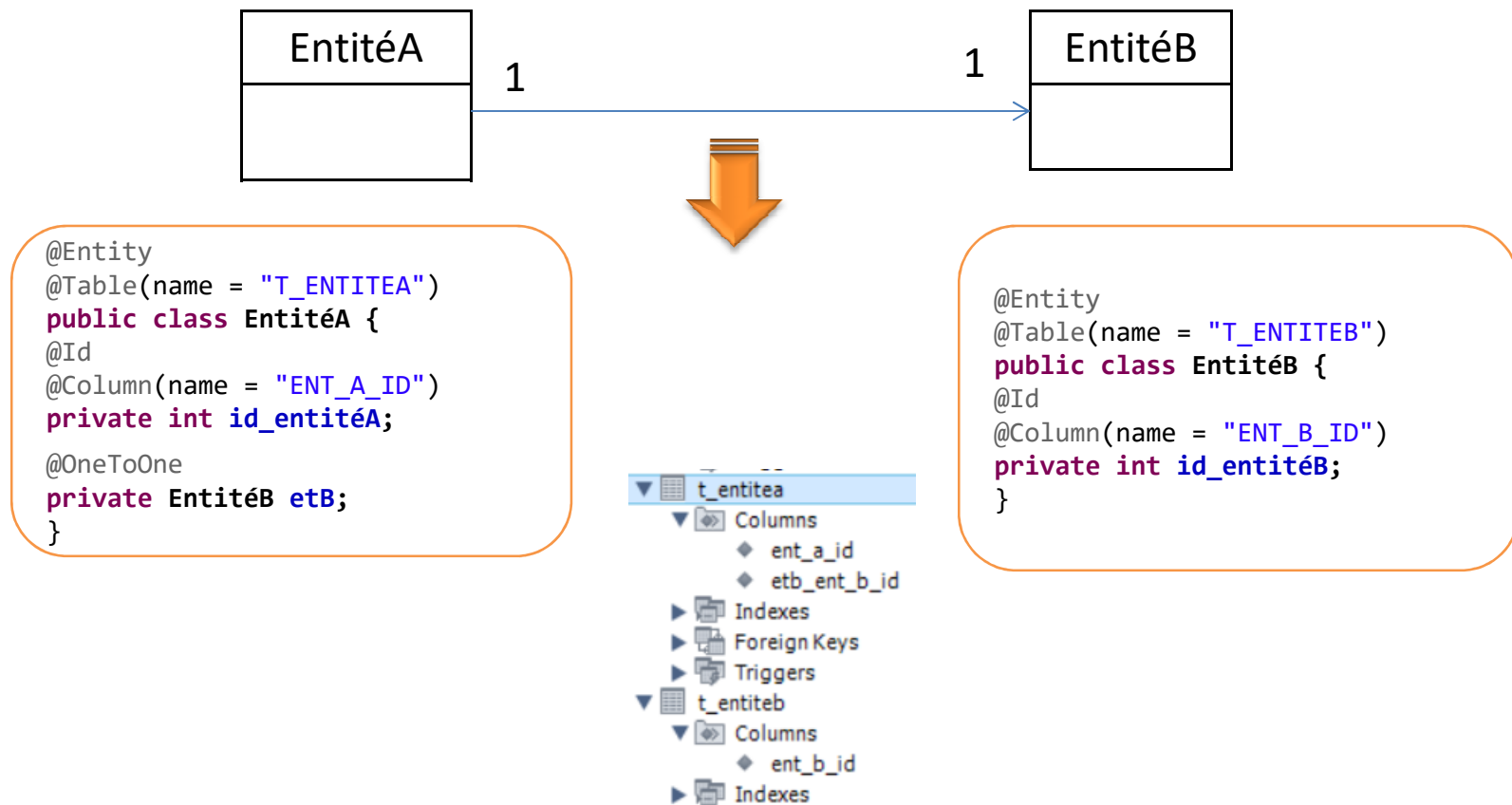


Partie 3 : Associations

Associations

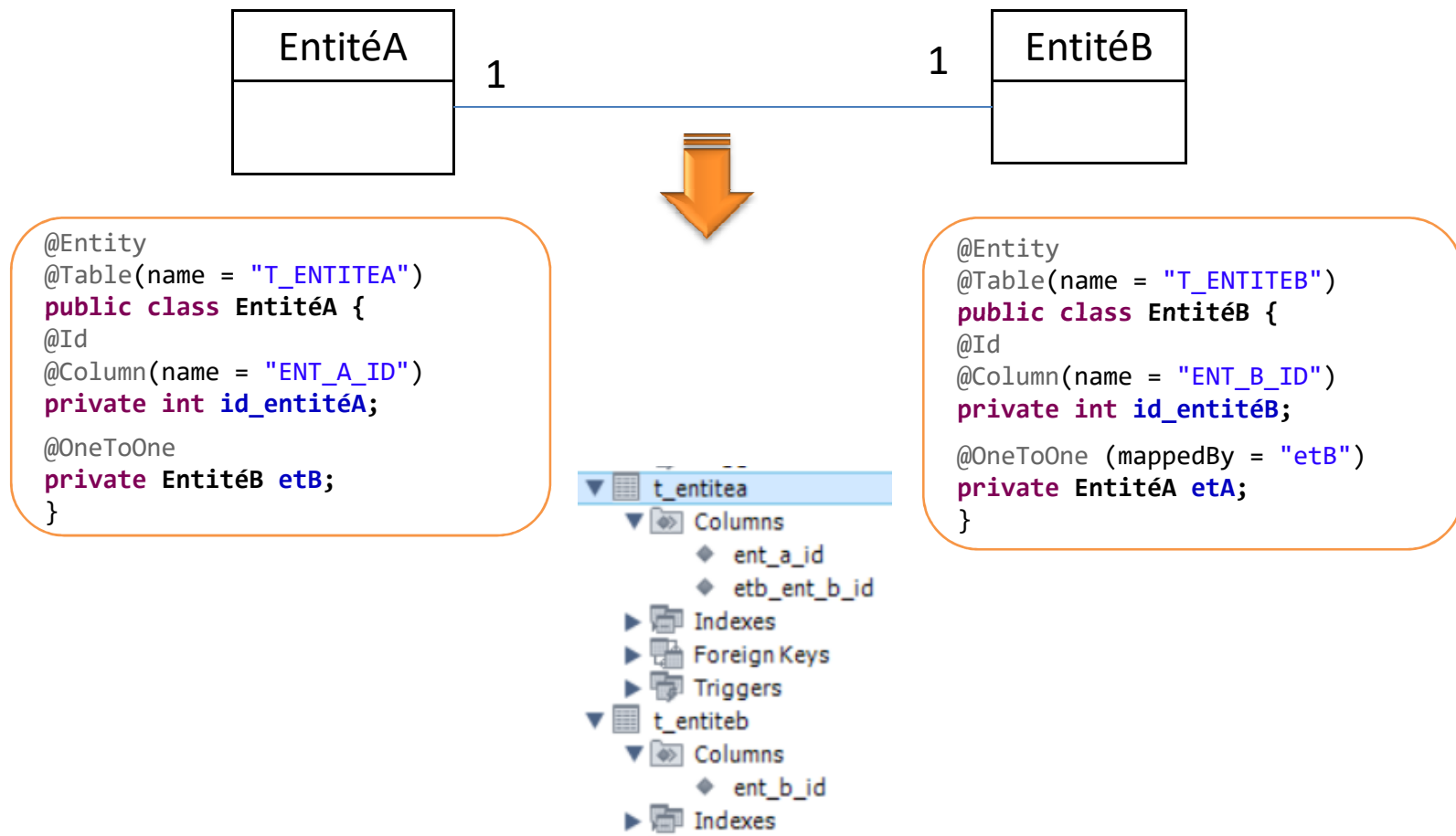
□ Association One-to-One

○ Unidirectionnelle



Associations

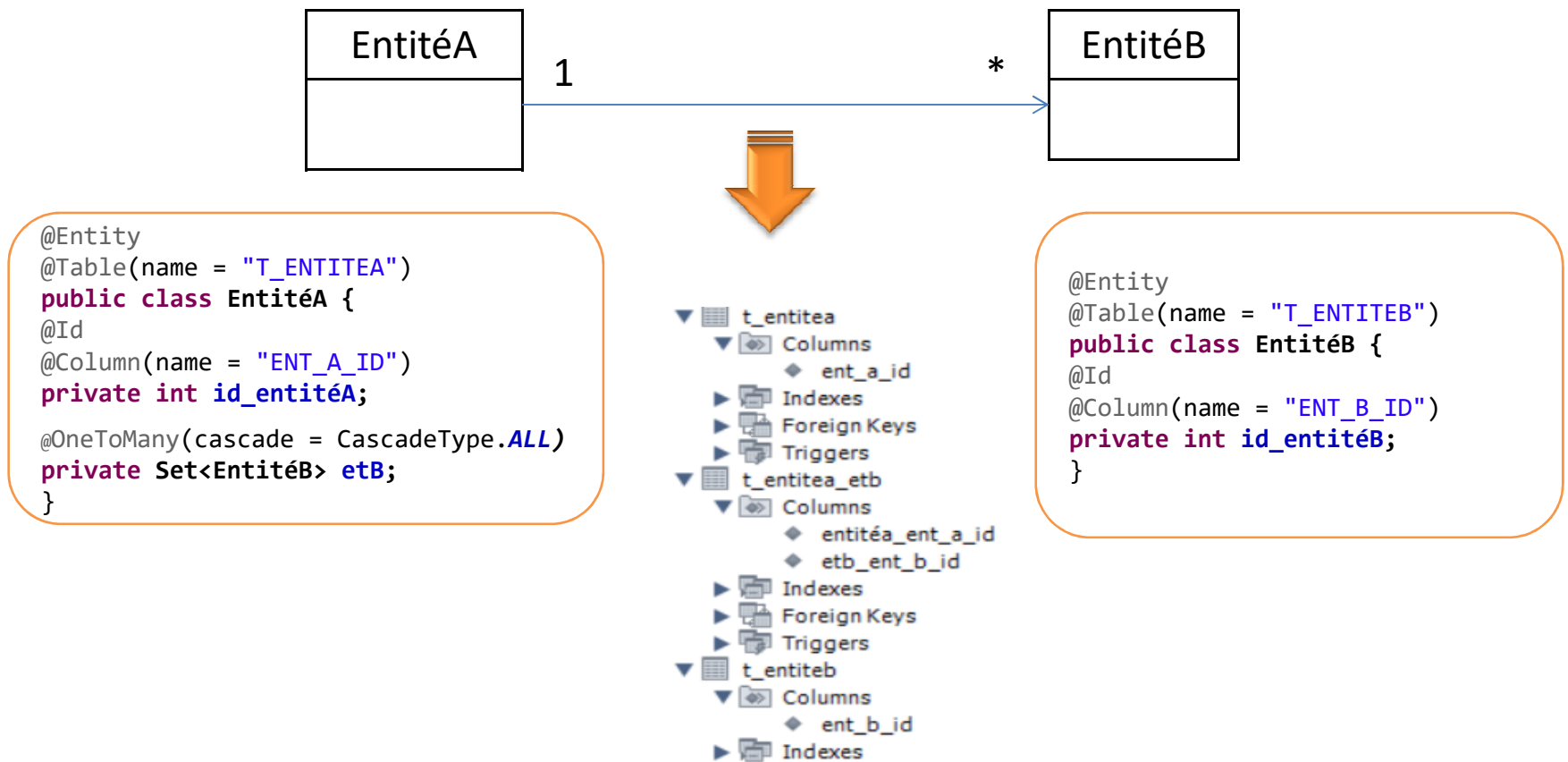
○ Bidirectionnelle



Associations

□ Association One-to-Many

○ Unidirectionnelle



Associations

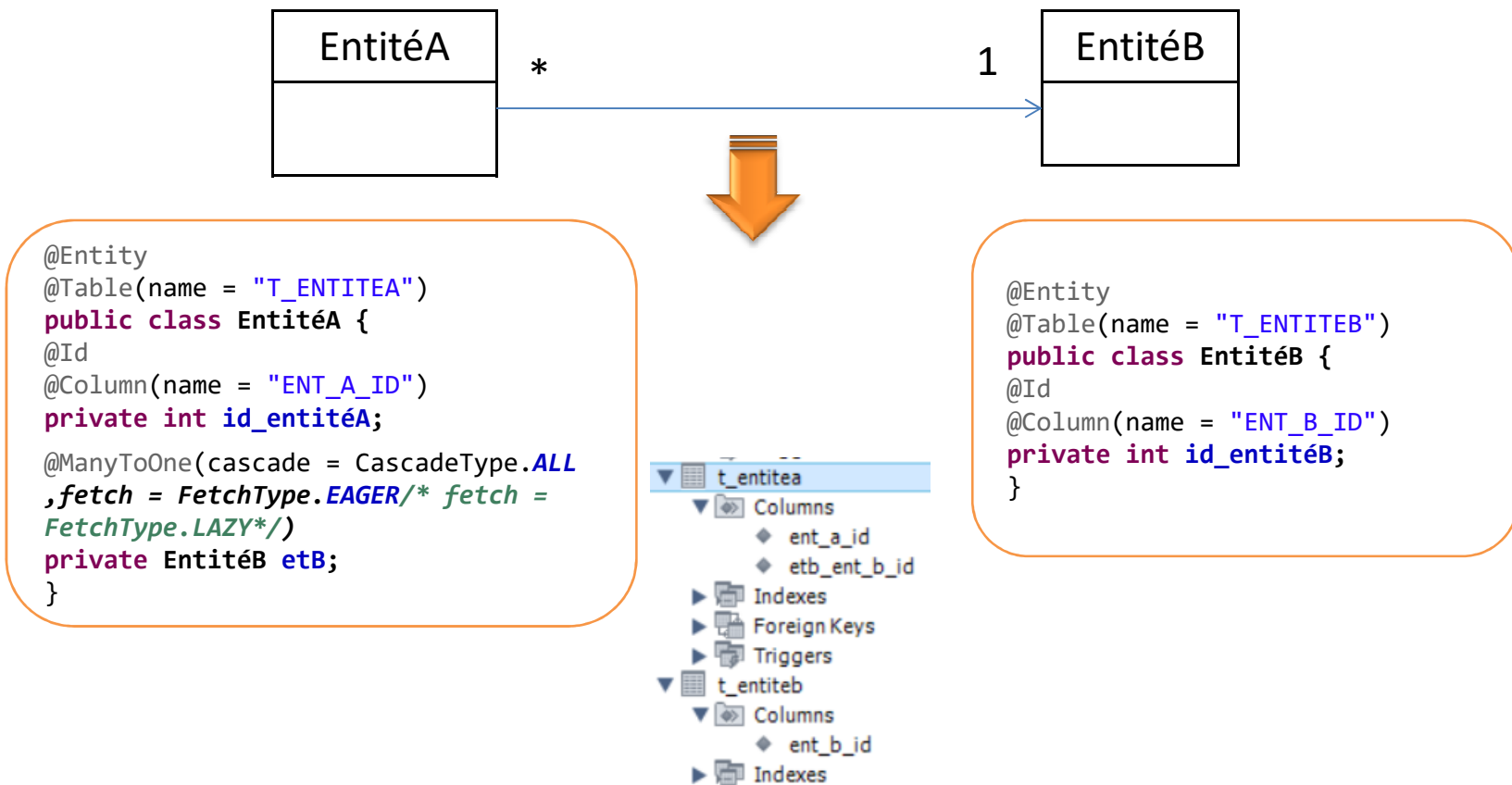
- *Bidirectionnelle*

One To Many Bidirectionnelle $\equiv$ Many To One Bidirectionnelle

Associations

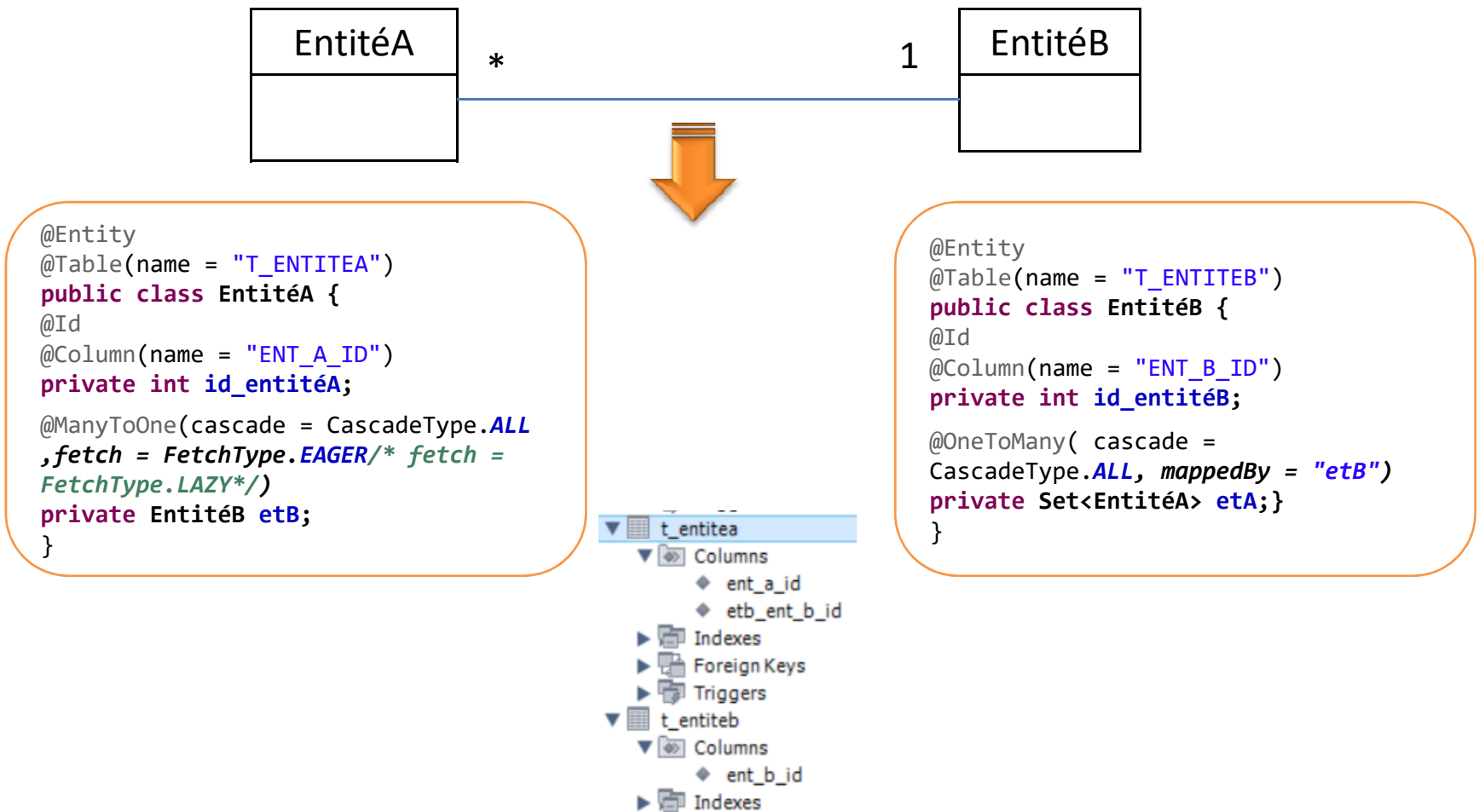
□ Association Many-to-One

○ Unidirectionnelle



Associations

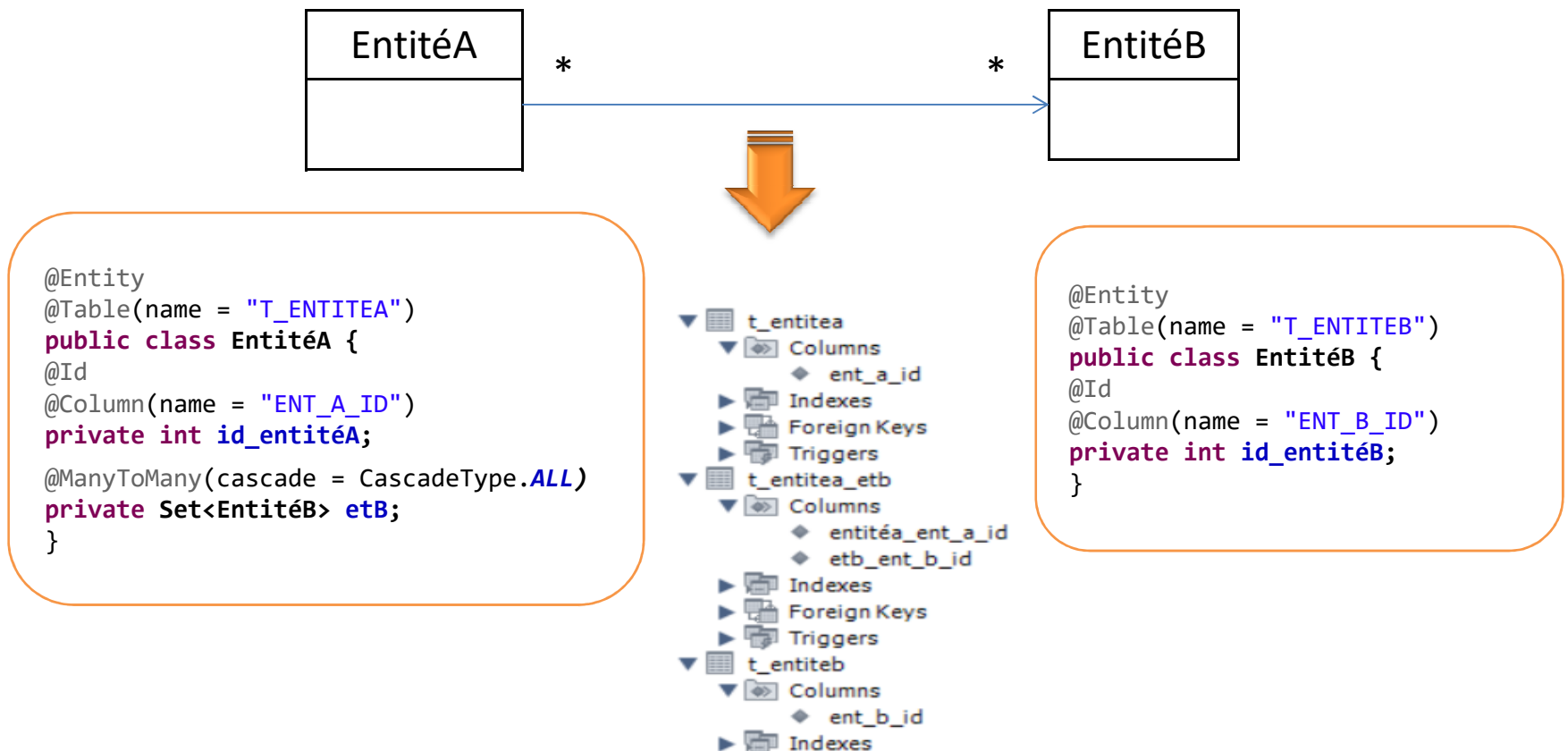
○ Bidirectionnelle



Associations

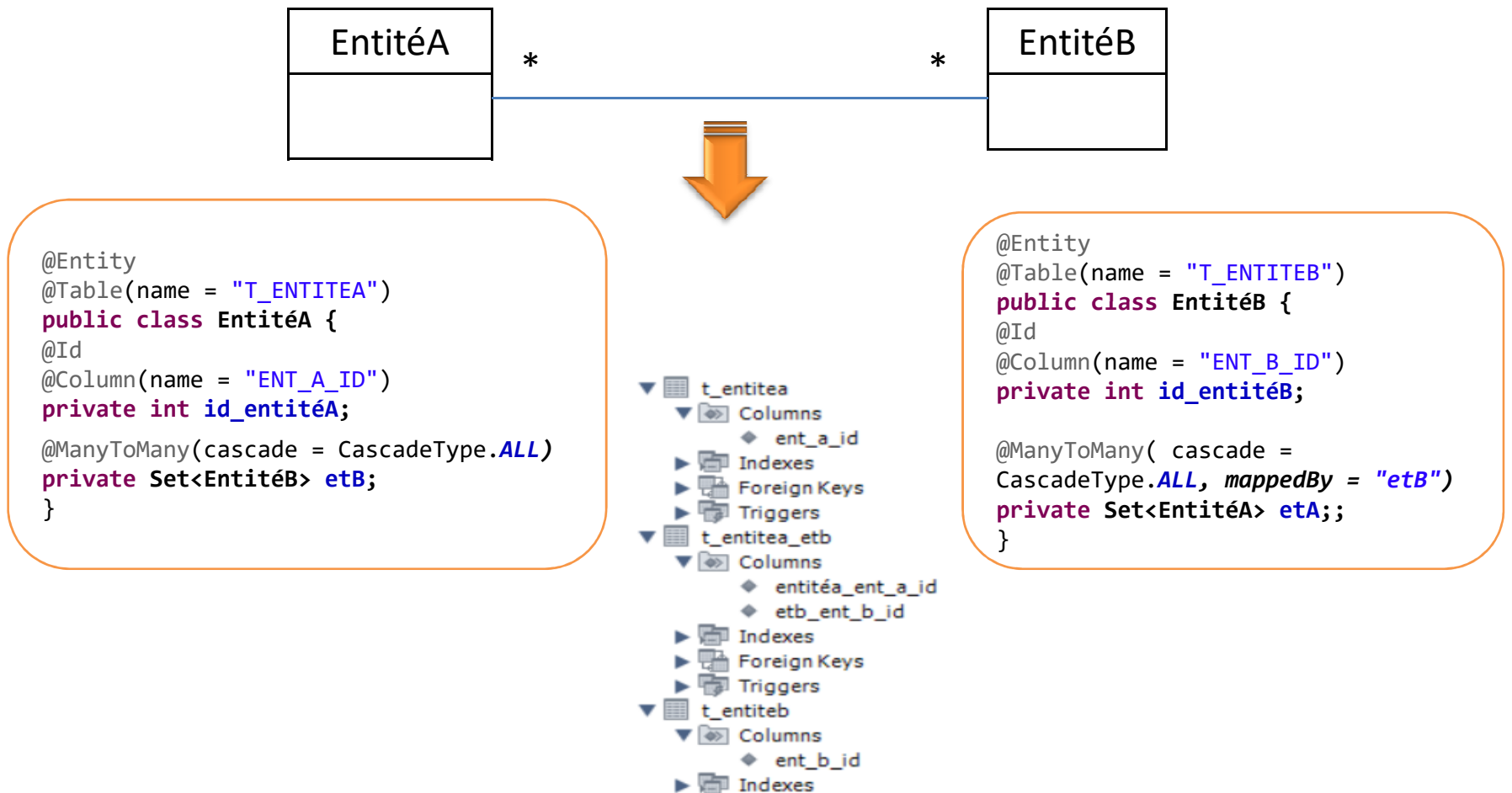
□ Association Many-to-Many

○ Unidirectionnelle



Associations

0 Bidirectionnelle



Associations

□ Fetch / Cascade

○ Fetch

- Permet de préciser comment la propriété est chargée selon deux modes :
 - **LAZY** : la valeur est chargée uniquement lors de son utilisation
 - **EAGER** : la valeur est toujours chargée (valeur par défaut)

○ Cascade

- Permet de préciser le type de cascade des opérations

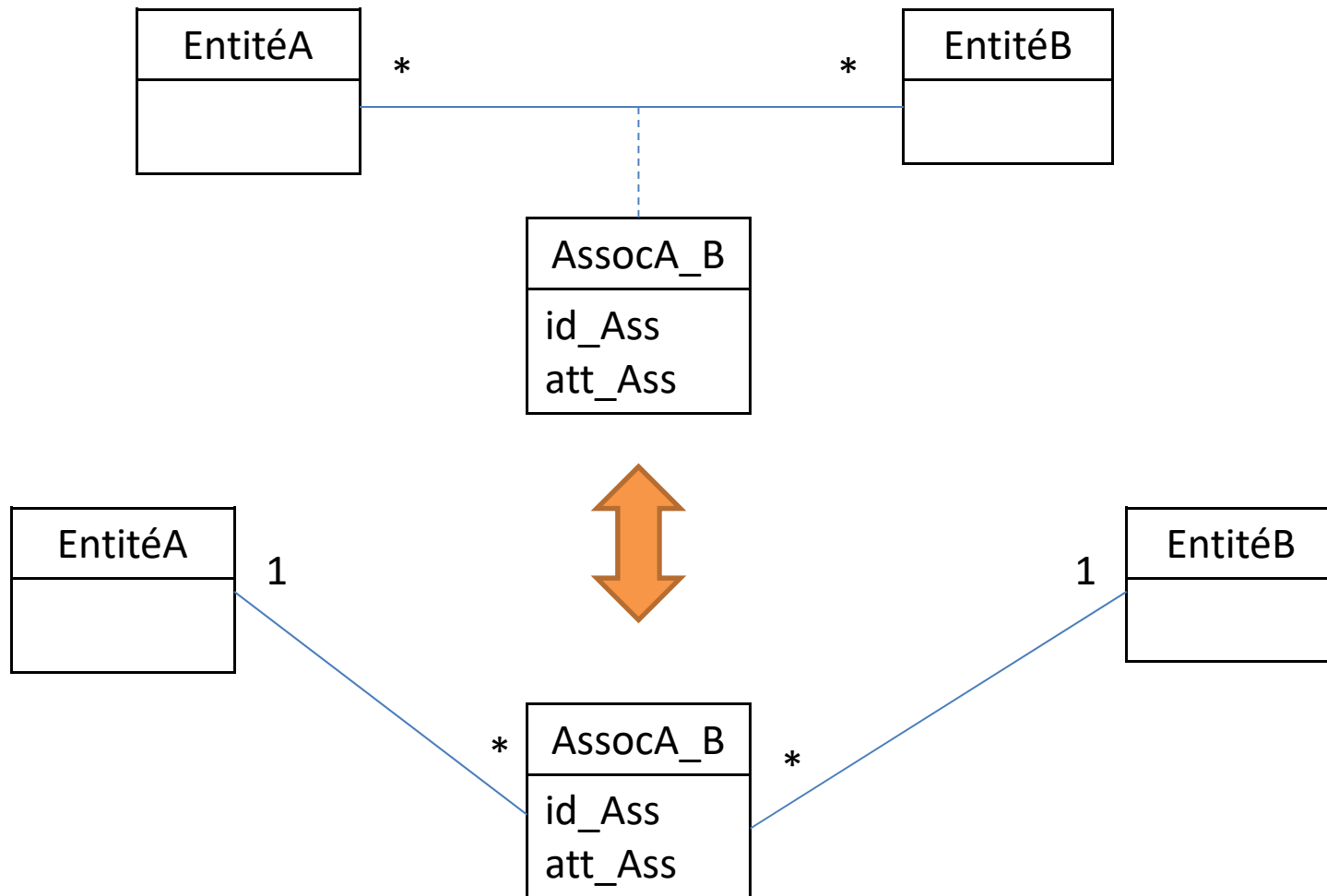
```
@ManyToMany(cascade = CascadeType.ALL, mappedBy =  
private
```

Cascade all operations

- ALL : CascadeType - CascadeType
- DETACH : CascadeType - CascadeType
- MERGE : CascadeType - CascadeType
- PERSIST : CascadeType - CascadeType
- REFRESH : CascadeType - CascadeType
- REMOVE : CascadeType - CascadeType
- class : Class<javax.persistence.CascadeType>

Associations

□ Association Many-to-Many Porteuse de Données



Associations



```
@Entity
@Table(name = "T_ENTITEA")
public class EntitéA {
    @Id
    @Column(name = "ENT_A_ID")
    private int id_entitéA;

    @OneToMany(mappedBy = "etA")
    private List<AssocA_B> associationsA;
}
```

```
@Entity
@Table(name = "T_ENTITEB")
public class EntitéB {
    @Id
    @Column(name = "ENT_B_ID")
    private int id_entitéB;

    @OneToMany(mappedBy = "etB")
    private List<AssocA_B> associationsB;
}
```

```
@Embeddable
public class AssocA_B_PK implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id_entitéA;
    private int id_entitéB;
    private int id_Ass;
    //...
}
```

```
@Entity
public class AssocA_B implements Serializable {
    private static final long serialVersionUID = 1L;
```

```
@EmbeddedId
private AssocA_B_PK assocA_B_PK;

private String att_Ass;
```

```
@ManyToOne
@JoinColumn(name = "id_entitéA", referencedColumnName =
"ENT_A_ID", insertable = false , updatable = false)
private EntitéA etA;
```

```
@ManyToOne
@JoinColumn(name = "id_entitéB", referencedColumnName =
"ENT_B_ID", insertable = false , updatable = false)
private EntitéB etB;
```

```
//...
}
```

Associations



▼ Tables										
▼ assoc_a_b										
▼ Columns										
◆ id_ass										
◆ id_entitéa										
◆ id_entitéb										
◆ att_ass										
▶ Indexes										
▶ Foreign Keys										
▶ Triggers										
▶ hibernate_sequences										
▼ t_entitea										
▼ Columns										
◆ ent_a_id										
▶ Indexes										

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id_ass	INT(11)	☒	☒	☐	☐	☐	☐	☐	☐	
id_entitéa	INT(11)	☒	☒	☐	☐	☐	☐	☐	☐	
id_entitéb	INT(11)	☒	☒	☐	☐	☐	☐	☐	☐	
att_ass	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
		☐	☐	☐	☐	☐	☐	☐	☐	



Partie 4 : Repository

Repository

□ Concept

- Spring Data JPA Fournit l'interface *JpaRepository<T, ID>* qui hérite de *CrudRepository<T, ID>* et qui fournit un ensemble de méthodes plus spécifiquement adaptées pour interagir avec une base de données relationnelle.
- L'interface *CrudRepository<T, ID>* hérite de *Repository<T, ID>* et fournit un ensemble d'opérations élémentaires pour la manipulation des objets.
- il suffit de définir une interface qui permet de manipuler une entité, en étendant l'interface *CrudRepository* et de déclarer les méthodes pour manipuler cette entité.

Repository

□ CrudRepository

```
public interface CrudRepository<T, ID extends Serializable>
    extends Repository<T, ID> {

    <S extends T> S save(S entity); ❶

    T findOne(ID primaryKey);        ❷

    Iterable<T> findAll();           ❸

    Long count();                    ❹

    void delete(T entity);           ❺

    boolean exists(ID primaryKey);   ❻

    // ... more functionality omitted.
}
```

- ❶ Saves the given entity.
- ❷ Returns the entity identified by the given id.
- ❸ Returns all entities.
- ❹ Returns the number of entities.
- ❺ Deletes the given entity.
- ❻ Indicates whether an entity with the given id exists.

Repository

□ Création [Exemples]

```
interface UserRepository extends CrudRepository<User, Long> {  
    long countByLastname(String lastname);  
}
```

```
public interface UserRepository extends Repository<User, Long> {  
    List<User> findByEmailAddressAndLastname(String emailAddress, String lastname);  
}
```

```
public interface UserRepository extends JpaRepository<User, Long> {  
    @Query("select u from User u where u.firstname like %?1")  
    List<User> findByFirstnameEndsWith(String firstname);  
}
```

- *Spring Data JPA va automatiquement créer une classe qui implémente cette interface au moment de l'exécution de la l'application.*

Repository

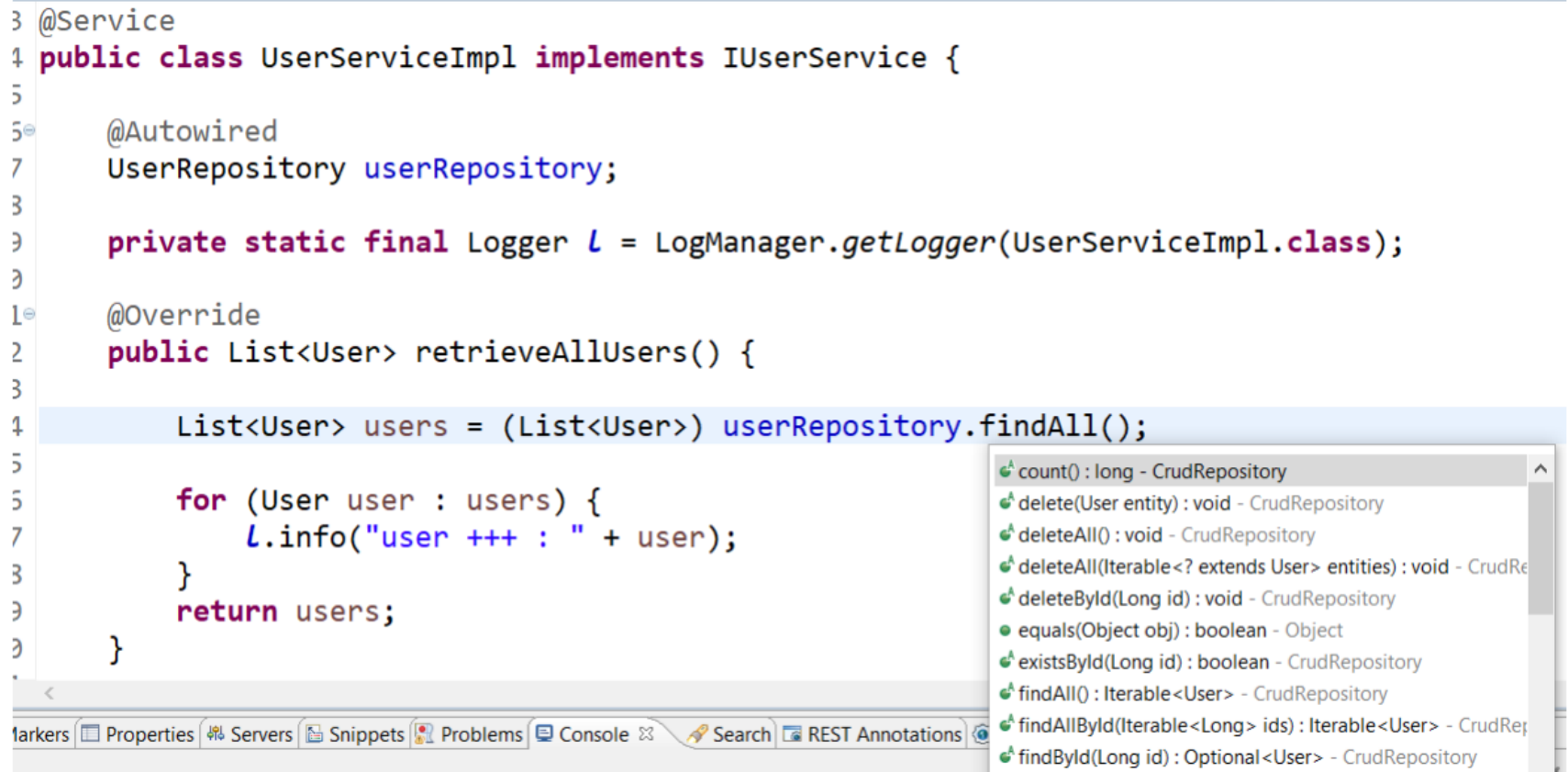
□ Méthodes[Exemples]

Keyword	Sample	Equivalent to
GreaterThan	<code>findByAgeGreaterThan(int age);</code>	Select u from User u where u.age > :age
LessThan	<code>findByAgeLessThan(int age);</code>	Select u from User u where u.age < :age
Between	<code>findByAgeBetween(int from, int to);</code>	Select u from User u where u.age between :from and :to
IsNotNull, NotNull	<code>findByFirstnameNotNull();</code>	Select u from User u where u.firstname is not null
IsNull, Null	<code>findByFirstnameNull();</code>	Select u from User u where u.firstname is null
Like	<code>findByFirstnameLike(String name);</code>	Select u from User u where u.firstname like :name
(No keyword)	<code>findByFirstname(String name);</code>	Select u from User u where u.firstname = :name
(No keyword)	<code>findOne(ID primaryKey);</code>	Select u from User u where u.id =:primaryKey
(No keyword)	<code>Long count();</code>	Select count(*) From User

Repository

□ Exemple d'Utilisation

```
3 @Service
4 public class UserServiceImpl implements IUserService {
5
6     @Autowired
7     UserRepository userRepository;
8
9     private static final Logger l = LogManager.getLogger(UserServiceImpl.class);
10
11     @Override
12     public List<User> retrieveAllUsers() {
13
14         List<User> users = (List<User>) userRepository.findAll();
15
16         for (User user : users) {
17             l.info("user +++ : " + user);
18         }
19         return users;
20     }
21 }
```



The screenshot shows an IDE window with a Java class `UserServiceImpl` implementing `IUserService`. The class is annotated with `@Service` and `@Autowired`. It has a `UserRepository` field and a `retrieveAllUsers()` method. The `retrieveAllUsers()` method calls `userRepository.findAll()`. A dropdown menu is open, showing the methods of `CrudRepository`. The methods listed are:

- `count() : long - CrudRepository`
- `delete(User entity) : void - CrudRepository`
- `deleteAll() : void - CrudRepository`
- `deleteAll(Iterable<? extends User> entities) : void - CrudRepository`
- `deleteById(Long id) : void - CrudRepository`
- `equals(Object obj) : boolean - Object`
- `existsById(Long id) : boolean - CrudRepository`
- `findAll() : Iterable<User> - CrudRepository`
- `findAllById(Iterable<Long> ids) : Iterable<User> - CrudRepository`
- `findById(Long id) : Optional<User> - CrudRepository`



Partie 5 :

JPQL

JPQL

□ Présentation

- JPQL : *Java Persistence Query Language*.
- JPQL est un langage de requête orienté objet défini par la spécification JPA qui permet de gérer la persistance de données dans des applications java.
- JPQL peut être considéré comme une version orientée objet de SQL.
- Il permet d'exercer des requêtes sur des **JPA Entity** mappées dans des tables de bases de données relationnelles .
- Il permet de récupérer des informations en utilisant la clause SELECT, et d'effectuer des mises à jour à l'aide de la clause UPDATE et la clause DELETE.

□ Annotations

- *@Query*: permet de définir une requête personnalisée que ce soit en utilisant SQL ou JPQL.
- *@Param*: cette annotation permet de spécifier les paramètres à passer à la requête.
- *@Modifying* : cette annotation permet d'activer la modification dans la base de données.

JPQL

□ Exemples

○ Sélection (Select)

• JPQL :

```
@Query("SELECT u FROM User u WHERE u.role= :role")  
List<User> retrieveUsersByRole(@Param("role") Role role);
```

C'est équivalent à :

```
@Query("SELECT u FROM User u WHERE u.role= ?1")  
List<User> retrieveUsersByRole(Role role);
```

• Native Query (SQL et non JPQL) :

```
@Query(value = "SELECT * FROM T_USER u WHERE u.role= :role " , nativeQuery =  
true)
```

```
List<User> retrieveUsersByRole(@Param("role") Role role);
```

C'est équivalent à :

```
@Query(value = "SELECT * FROM T_USER u WHERE u.role= ?1 " , nativeQuery = true)  
List<User> retrieveUsersByRole(Role role);
```

JPQL

○ *Mise à Jour (Update)*

JPQL :

@Modifying

```
@Query("update User u set u.role = :role where u.firstName = :fname")  
int updateUserStatusByFirstName(@Param("role") Role role, @Param("fname") String  
    fname);
```

○ *Suppression (Delete)*

JPQL :

@Modifying

```
@Query("DELETE FROM User u WHERE u.role = :role AND u.firstName = :fname")  
int deleteUserByStatusAndFirstName(@Param("role") Role role, @Param("fname")  
    String fname);
```

JPQL

o *Insertion (Insert)*

- INSERT n'est pas prise en considération par les spécifications JPA

==> Pas de JPQL pour les requêtes INSERT.

==> Utiliser les Natives Query.

Native Query (SQL et non JPQL) :

@Modifying

```
@Query(value = "INSERT INTO T_USER (firstName, lastName, role) VALUES (:fn, :ln, :role)", nativeQuery = true)
```

```
void insertUser(@Param("fn") String fn, @Param("ln") Integer ln, @Param("role") Integer role);
```



Spring

TP

